

黄佳 - ClaudeCode实践 - 第8章 器以载道：Agent SDK智能体开发套件

借舟者随波而行，造舟者定其去向。

自从小雪将Claude Code集成至公司的CI/CD流水线，实现了自动审查PR与生成报告后，团队反响热烈。受此启发，产品经理提出了一个更具前瞻性的构想。

“我们能否开发一款内部工具，让非技术背景的同事也能利用Claude Code分析代码？例如，产品经理只需要输入用户故事，系统便能自动定位相关代码模块、评估实现复杂度，并列待修改的文件清单。”

小冰稍作沉吟。虽然Headless模式能部分满足需求，但它本质上是一个命令行工具，对不熟悉终端操作的产品经理而言门槛过高。他们真正需要的是一款具备表单输入、进度反馈及历史记录功能的Web界面。这意味着，这已不再是单纯的“工具使用”，而是需要“构建一款产品”。

“Headless模式通过参数驱动Claude，而Agent SDK则通过代码编排Claude，”咖哥补充道，“两者的区别，好比‘使用计算器’与‘为计算器编程’：前者只能依赖预设的功能按钮，后者则能自定义其行为逻辑。”

8.1 Agent SDK的定位：从工具到组件

本书前面介绍的机制（包括CLAUDE.md、子智能体、Skills、Hooks、MCP以及Headless模式）均以“使用Claude Code”为核心视角。在这一范式下，开发者作为用户，在Claude Code既定的框架内通过配置来扩展其行为边界。

Agent SDK的出发点则截然不同。它预设开发者是应用构建者，旨在将Claude Code的核心能力深度嵌入自有应用中。何时启动Claude、传入何种参数、如何编排中间执行步骤，以及如何管理多轮对话的状态，这一切完全由开发者的代码自主控制。此时，你不再是在“使用”一个现成的产品，而是在“构建”一个以Claude为智能引擎的全新应用。

这一演进路径如图8-1所示，该图清晰地展示了控制粒度从“配置”向“代码”的跨越。



(图8-1 从CLAUDE.md到Agent SDK的控制粒度演进路径)

每深入一层，控制粒度便愈发精细，定制化程度也随之提升。Agent SDK处于这一演进链的末端：它在赋予开发者最大灵活性的同时，也带来了最高的工程复杂度。

Agent SDK提供Python与TypeScript/JavaScript两个版本，两者功能完全对等。值得注意的是，自2025年末起，其包名已由claude-code-sdk正式更名为claude-agent-sdk，以更准确地反映其定位。安装方式如下。

```
# Python
pip install claude-agent-sdk

# TypeScript/Node.js
npm install @anthropic-ai/claude-agent-sdk
```

8.2 核心API：query函数

Agent SDK的入口是query函数。如果说Headless模式的入口是`claude -p "prompt"`，那么query函数便是其在编程层面的等价物--不过，它返回的并非最终结果，而是一个**异步消息流**。

8.2.1 Python版本

以下代码展示了如何使用Agent SDK通过Python以编程方式调用Agent，执行分析代码库架构的任务。

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def analyze_code():
    options = ClaudeAgentOptions(
        max_turns=5,
        allowed_tools=["Read", "Grep", "Glob"],
        system_prompt="你是一名代码架构分析师",
    )

    async for message in query(
        prompt="分析 src/auth/ 目录的实现架构",
        options=options
    ):
        if message.type == "assistant":
            for block in message.content:
                if hasattr(block, 'text'):
                    print(block.text, end="", flush=True)
        elif message.type == "result":
            print(f"\n\n费用: ${message.total_cost_usd:.4f}")

asyncio.run(analyze_code())
```

8.2.2 TypeScript版本

以下代码展示了如何使用Agent SDK通过TypeScript以编程方式调用Agent，执行分析代码库架构的任务。

```
import { query, ClaudeAgentOptions } from '@anthropic-ai/claude-agent-sdk';

const options: ClaudeAgentOptions = {
  maxTurns: 5,
  allowedTools: ['Read', 'Grep', 'Glob'],
  systemPrompt: '你是一名代码架构分析师。',
};

async function analyzeCode() {
  for await (const message of query({
    prompt: '分析src/auth/目录的实现架构',
    options,
  })) {
    if (message.type === 'assistant') {
      for (const block of message.content) {
        if ('text' in block) {
          process.stdout.write(block.text);
        }
      }
    } else if (message.type === 'result') {
      console.log(`\n\n费用: ${message.totalCostUsd}`);
    }
  }
}

analyzeCode();
```

调用query函数会返回一个**异步生成器**(async generator)。它不需要等待Claude完成全部任务才返回，而是当Claude生成一条消息时，就即时返回该消息。在Web应用中，这意味着你可以将Claude的思考过程以流式形式推送至前端，复现类似Claude.ai的打字机效果；而在CI/CD场景中，它允许你在Claude仍在运行时就可以着手处理已生成的输出。

8.3 消息类型：解读Claude的输出流

“理解消息类型是使用Agent SDK的基石，”咖哥指出，“正如HTTP开发者必须精通状态码，Agent SDK开发者也需要透彻掌握每类消息的含义。”

query函数生成的消息流是一个有序序列，其中每条消息均通过type字段标识其具体类型。

8.3.1 system/init——会话初始化

以下代码展示了会话初始化的系统消息示例。

```
{
  "type": "system",
  "subtype": "init",
  "session_id": "550e8400-...",
  "model": "claude-sonnet-4-6",
  "tools": ["Read", "Grep", "Glob"],
  "mcp_servers": []
}
```

消息流中的首条消息用于汇报当前会话的配置详情，涵盖模型版本、可用工具集及MCP服务器状态。这在调试过程中尤为关键：如果发现Claude未按预期调用某项工具，应首先核查init消息中是否已包含该工具的定义。

8.3.2 assistant——Claude的响应

以下代码展示了Claude每一轮响应的消息示例。

```
{
  "type": "assistant",
  "message": {
    "role": "assistant",
    "content": [
      {"type": "text", "text": "让我先看看目录结构....."},
      {"type": "tool_use", "id": "toolu_xxx", "name": "Glob",
"input": {"pattern": "src/auth/**/*.*ts"}}
    ]
  }
}
```

在上述代码中，content字段是一个数组，可混合包含文本块（即Claude的“思考”过程或最终输出）与工具调用块（即Claude决定执行的操作）。值得注意的是，单条assistant消息中可能包含多个工具调用，这意味着Claude能够并发请求读取多个文件或执行其他并行操作。

8.3.3 user——工具执行结果

以下代码展示了工具执行完成后的结果示例。

```
{
  "type": "user",
  "message": {
    "role": "user",
    "content": [
      {"type": "tool_result", "tool_use_id": "toolu_xxx",
"content": "src/auth/\n├─ login.ts\n├─ session.ts\n└─ middleware.ts"}
    ]
  }
}
```

```

    ]
  }
}

```

请注意，tool_use_id需要与assistant消息中的工具调用id严格对应。该映射关系由Agent SDK自动维护，开发者通常不需要手动处理。

8.3.4 result--任务完成

以下代码展示了任务完成时的终止消息示例。

```

{
  "type": "result",
  "subtype": "success",
  "session_id": "550e8400-...",
  "num_turns": 5,
  "total_cost_usd": 0.0342,
  "duration_ms": 12345,
  "duration_api_ms": 10000,
  "usage": {
    "input_tokens": 5000,
    "output_tokens": 1500,
    "cache_read_input_tokens": 3000
  },
  "result": "架构分析: src/auth/ 采用分层设计...",
  "structured_output": null
}

```

在上述代码中，subtype是判断任务结束状态的关键字段（详见表8-1）。

表8-1 result消息的subtype类型与含义

以下代码展示了一个实用的消息处理模式，用于在异步消息流中收集文本输出、记录工具调用详情并存储会话元数据。

```

async def process_query(prompt: str, options: ClaudeAgentOptions) -> dict:
    # 初始化结果容器
    result = {"text": [], "tools": [], "metadata": {}, "error": None}

    # 异步遍历消息流
    async for message in query(prompt=prompt, options=options):
        if message.type == "assistant":
            # 处理助手响应中的内容块
            for block in message.content:
                if hasattr(block, 'text'):

```

```

        # 收集文本输出（思考或回答）
        result["text"].append(block.text)
    elif hasattr(block, 'name'):
        # 记录工具调用（名称与输入参数）
        result["tools"].append({"tool": block.name, "input":
block.input})
    elif message.type == "result":
        # 提取终止消息中的元数据
        result["metadata"] = {
            "session_id": message.session_id,
            "cost": message.total_cost_usd,
            "turns": message.num_turns,
            "duration_ms": message.duration_ms,
        }
        # 检查是否因错误终止
        if message.is_error:
            result["error"] = message.subtype

    return result

```

8.4 ClaudeAgentOptions：精细的行为控制

ClaudeAgentOptions是控制ClaudeAgent行为的核心配置对象（该类名已由旧版本的Claude CodeOptions更新而来）。它将Headless CLI的命令行参数转化为编程接口，使得动态配置和精细化控制成为可能。

以下代码展示了对该配置对象的结构解析。

```

from claude_agent_sdk import ClaudeAgentOptions

options = ClaudeAgentOptions(
    # === 模型与执行控制 ===
    model="claude-sonnet-4-6",          # 指定使用的模型版本
    max_turns=10,                       # 限制最大交互轮数，防止死循环
    max_budget_usd=1.0,                 # 设置单次任务的费用上限（美元），超出即停止

    # === 工具权限管理 ===
    # 白名单：仅允许使用列出的工具
    allowed_tools=["Read", "Grep", "Glob", "Write"],
    # 黑名单：明确禁止使用的工具（优先级通常高于白名单或作为补充）
    disallowed_tools=["Bash"],

    # === 权限模式 ===
    # default：标准模式，需用户确认危险操作
    # acceptEdits：自动接受文件编辑

```

```

# plan: 仅规划不执行
# bypassPermissions: 跳过所有确认（高风险，仅限受信任环境）
permission_mode="default",

# === 提示工程 ===
system_prompt="你是一名高级代码审查员",      # 设定核心系统指令
append_system_prompt="务必检查SQL注入漏洞", # 在核心指令后追加特定要求

# === 工作环境配置 ===
cwd="/path/to/project",      # 设置Agent的工作目录
env={"PROJECT_NAME": "MyApp"}, # 注入自定义环境变量

# === 会话管理 ===
resume="session-id-to-resume", # 从指定的session_id恢复上下文
no_session_persistence=False,  # 若为True，则任务结束后不保存会话历史

# === 结构化输出 ===
# 强制模型输出符合特定JSON Schema格式的数据
output_format={"type": "json_schema", "schema": my_schema}, # 结构化输出

# === MCP集成 ===
# 动态启动并连接外部MCP服务器
mcp_servers=[
    {"name": "db", "command": "python", "args": ["/db_server.py"]}
],
)

```

8.4.1 权限模式详解

权限模式是Agent SDK应用安全防线的核心。它决定了Claude在遇到潜在风险操作（如写文件、执行命令）时的行为逻辑（详见表8-2）。

表8-2 Agent SDK权限模式与适用场景

模式	行为逻辑	适用场景
default	标准交互。遇到危险操作(如写文件、运行Shell)时，等待外部确认(通常通过回调或人工干预)	面向用户的Web应用、交互式CLI工具
acceptEdits	半自动。自动接受文件编辑类操作，但对于执行命令(Bash)、网络请求等仍会询问	自动化修复脚本、代码格式化流水线
plan	只读/规划模式。禁止所有修改。模型只能读取文件、分析代码并输出计划，无法执行写操作或命令	代码审查、架构分析、漏洞扫描

模式	行为逻辑	适用场景
bypassPermissions	完全信任。跳过所有权限检查，自动执行所有操作(包括rm -rf、curl等高危命令)	严格隔离的CI容器、沙箱环境

“在CI/CD流水线中，bypassPermissions看似便捷——毕竟不需要人工确认嘛，”咖哥说，“但请务必牢记，这意味着赋予Claude执行任意命令的权限，包括删除文件、修改配置，甚至发起网络请求。因此，仅在确保执行环境完全隔离（如容器化部署、文件系统只读且无网络连接）时，方可启用该选项。”

8.4.2 工具权限的模式匹配

allowed_tools支持与Headless CLI相同的模式匹配语法。请看以下代码示例。

```
options = ClaudeAgentOptions(
    allowed_tools=[
        "Read",
        "Grep",
        "Glob",
        "Bash(git diff *)",          # 仅允许执行git diff命令
        "Bash(npm test *)",        # 仅允许执行npm test命令
        "mcp__database__query",    # 仅允许使用MCP数据库查询工具
    ]
)
```

8.5 会话管理：跨调用的上下文延续

query函数适用于单次任务场景。然而，许多实际应用需要多轮交互：用户首先指示Claude分析问题，待结果返回后再指定修复方向，最后在修复完成后要求Claude进行验证。

8.5.1 基于session_id的会话延续

以下代码展示了如何使用一个支持会话延续的异步Agent接口，分两步完成一个复杂的**安全分析**任务。

```
# 第一次调用：分析问题
session_id = None
async for message in query(prompt="分析 src/auth 的安全问题", options=options):
    if message.type == "system" and message.subtype == "init":
        session_id = message.session_id  # 【关键】获取会话ID
    if message.type == "result":
        print(message.result)

# 第二次调用：在上一轮上下文的基础上继续深入
```



```

resume_options = ClaudeAgentOptions(**options.__dict__, resume=session_id)
async for message in query(
    prompt="重点分析你发现的第一个SQL注入风险",
    options=resume_options
):
    if message.type == "result":
        print(message.result)

```

resume参数使Claude能够“记忆”上一轮对话的完整上下文（包括已读取的文件、执行的分析步骤以及得出的结论）。相较于在第二个Prompt中重复描述上下文，这种方式不仅效率高，而且能确保分析的准确性与连贯性。

8.5.2 会话分叉

以下代码展示了如何利用会话分叉技术，基于同一个初始分析结果，探索多个不同的解决方案或假设场景且确保各分支互不干扰。

```

# 基于同一个分析结果，探索不同的方向
# 创建分叉配置：复用原会话状态，但开启新分支
options_fork = ClaudeAgentOptions(**options.__dict__, resume=session_id,
fork_session=True)

# 方向 A：探讨微服务重构方案
async for message in query(
    prompt="如果重构为微服务架构，需要修改哪些部分？",
    options=options_fork
):
    ...

# 方向 B：探讨现有架构的安全加固方案
# 基于同一个 session_id 再次分叉，确保两个方向完全独立
async for message in query(
    prompt="如果保持现有架构，应如何加固安全性？",
    options=options_fork
):
    ...

```

设置fork_session=True可从指定会话的特定时间点“分叉”出一个全新的会话副本，而原始会话的状态保持不变。这一机制在需要进行“假设性分析”或对比多种技术路线的场景中极具价值。

8.6 自定义工具：扩展Claude的能力边界

Claude原生内置了Read、Write、Bash、Grep等基础工具，但在实际应用场景中，开发者往往需要集成特定领域的能力，如查询数据库、调用内部API、发送通知或执行复杂的业务逻辑。

Agent SDK中的自定义工具实质上是进程内MCP服务器。与第6章介绍的独立部署的MCP服务器不同，它不需要启动单独的进程，而是直接应用进程中运行。这种架构消除了进程间通信的开销，从而显著提升了工具调用的效率与响应速度。

8.6.1 使用@tool装饰器定义工具

以下代码展示了如何使用Agent SDK构建一台进程内MCP服务器，将两个具体的业务功能（数据库查询与发送通知）封装为安全的工具，供Agent调用。

```
from claude_agent_sdk import tool, create_sdk_mcp_server
import json

# 工具1: 数据库查询（只读）
@tool(
    name="query_database",
    description="Execute a read-only SQL query on the application database",
    parameters={
        "query": str,
        "limit": int
    }
)
async def query_database(args):
    sql = args["query"]
    limit = args.get("limit", 100)

    # 【安全关键】强制校验：仅允许SELECT语句
    if not sql.strip().upper().startswith("SELECT"):
        return {
            "content": [{"type": "text", "text": "Error: Only SELECT queries allowed"}],
            "isError": True
        }

    results = await db.execute(f"{sql} LIMIT {limit}")
    return {
        "content": [{"type": "text", "text": json.dumps(results, indent=2)}]
    }

# 工具2: 发送通知
@tool(
    name="send_notification",
    description="Send a notification to the team Slack channel",
    parameters={
        "channel": str,
        "message": str
    }
)
```

```

    }
)
async def send_notification(args):
    await slack.post_message(args["channel"], args["message"])

    return {
        "content": [{"type": "text", "text": f"Notification sent to #
{args['channel']}"}]
    }

# 创建MCP服务器承载这些工具
tools_server = create_sdk_mcp_server(
    name="app-tools",
    version="1.0.0",
    tools=[query_database, send_notification]
)

```

8.6.2 在Agent配置中注册并启用自定义工具

以下代码展示了如何在Agent配置中注册并启用自定义工具。

```

options = ClaudeAgentOptions(
    mcp_servers={
        "app-tools": tools_server
    },
    allowed_tools=[
        "Read", "Grep", "Glob",           # 内置工具
        "mcp__app-tools__query_database",  # 自定义工具
        "mcp__app-tools__send_notification",
    ]
)

```

自定义工具的名称严格遵循"`mcp{服务器名}{工具名}`"格式。这种带前缀的命名方式不仅避免了与内置工具（如Read、Bash）的名称冲突，更在日志审计和监控仪表盘中提供了天然的分类标签。

8.6.3 使用Pydantic模型进行参数验证

在处理复杂参数结构时，推荐使用Pydantic模型来替代简单的字典定义。请看以下代码示例。

```

from pydantic import BaseModel, Field
from claude_agent_sdk import tool

# 定义结构化参数模型
class DatabaseQueryParams(BaseModel):

```

```

    table: str = Field(
        ...,
        description="Table name"
    )
    columns: list[str] = Field(
        default=["*"],
        description="Columns to select"
    )
    where: str | None = Field(
        default=None,
        description="SQL WHERE clause condition (optional)"
    )
    limit: int = Field(
        default=100,
        ge=1,
        le=1000,
        description="Maximum number of rows to return"
    )

# 将模型直接作为工具参数定义
@tool(
    name="safe_query",
    description="Execute a safe, parameterized database query",
    parameters=DatabaseQueryParams
)
async def safe_query(args: DatabaseQueryParams):
    # args已经通过Pydantic模型验证，类型安全
    ...

```

咖哥发言

自定义工具的设计务必慎之又慎。Claude可能会出乎意料的方式调用工具，如传入极端参数值、在短时间内连续发起数十次调用，或利用前一次工具的输出构造新的输入。为此，请遵循以下三大原则：单一职责（确保每个工具仅专注于完成一个特定任务）、描述清晰（description字段是写给Claude看的“使用指南”，必须明确界定“什么时候可用”以及“什么时候禁用”）、零信任验证（永远不要信任Claude传入的任何参数，必须在工具内部对所有输入进行严格校验）。

8.7 Agent SDK中的Hooks：程序化的拦截

第5章介绍的Hooks机制依赖于JSON配置与Shell脚本；而在Agent SDK中，Hooks已全面升级为原生编程接口。开发者可以直接使用Python或TypeScript函数定义Hook逻辑，彻底摒弃了对Shell脚本和JSON配置文件的依赖。

8.7.1 PreToolUse：执行前拦截

以下代码展示了一个细粒度的运行时安全拦截系统。该系统利用Hooks机制，在Agent实际执行工具（如运行Bash命令或修改文件）之前，对操作进行深度检查与过滤。一旦检测到危险行为，系统将直接拒绝执行，从而有效防止AI造成破坏。

```
from claude_agent_sdk import ClaudeAgentOptions, HookMatcher

async def block_dangerous_bash(input_data, tool_use_id, context):
    """拦截危险的Bash命令"""
    # 仅处理Bash工具调用
    if input_data["tool_name"] != "Bash":
        return {}

    command = input_data["tool_input"].get("command", "")
    # 定义危险命令特征列表
    dangerous = ["rm -rf", "sudo", "chmod 777", "> /dev/", "mkfs", "dd if="]

    for pattern in dangerous:
        if pattern in command:
            return {
                "hookSpecificOutput": {
                    "hookEventName": "PreToolUse",
                    "permissionDecision": "deny",
                    "permissionDecisionReason": f"Blocked: {pattern}"
                }
            }
    return {}

async def protect_config_files(input_data, tool_use_id, context):
    """保护关键配置文件不被修改"""
    tool_name = input_data["tool_name"]
    # 仅处理文件写入类工具
    if tool_name not in ["Write", "Edit"]:
        return {}

    file_path = input_data["tool_input"].get("file_path", "")
    # 定义受保护的路径特征
    protected = [".env", "secrets", "production.yaml", "database/migrations"]

    for p in protected:
        if p in file_path:
            return {
                "hookSpecificOutput": {
                    "hookEventName": "PreToolUse",
                    "permissionDecision": "deny",
                    "permissionDecisionReason": f"Protected file: {file_path}"
                }
            }
```

```

        }
    return {}

# 配置Agent选项，注册安全拦截Hook
options = ClaudeAgentOptions(
    hooks={
        "PreToolUse": [
            HookMatcher(matcher="Bash", hooks=[block_dangerous_bash]),
            HookMatcher(matcher="Write", hooks=[protect_config_files]),
            HookMatcher(matcher="Edit", hooks=[protect_config_files]),
        ]
    }
)

```

8.7.2 PostToolUse：执行后处理

以下代码展示了如何利用PostToolUse（工具调用后）Hook实现自动化 workflow 增强与全链路审计。

```

import subprocess
import json
from datetime import datetime
from claude_agent_sdk import ClaudeAgentOptions, HookMatcher

async def auto_format_on_write(input_data, tool_use_id, context):
    """写入文件后自动触发代码格式化"""
    # 仅处理文件写入类工具
    if input_data["tool_name"] not in ["Write", "Edit"]:
        return {}

    file_path = input_data["tool_input"].get("file_path", "")

    # 根据文件扩展名选择对应的格式化工具
    if file_path.endswith(".py"):
        # 使用 Black 格式化 Python 代码
        subprocess.run(["black", file_path], capture_output=True)
    elif file_path.endswith((".ts", ".js", ".tsx", ".jsx")):
        # 使用 Prettier 格式化前端代码
        subprocess.run(["prettier", "--write", file_path],
            capture_output=True)

    return {}

async def audit_all_tools(input_data, tool_use_id, context):
    """记录所有工具调用的审计日志"""

```

```

    audit_entry = {
        "timestamp": datetime.now().isoformat(),
        "tool": input_data["tool_name"],
        "input": input_data["tool_input"],
        "tool_use_id": tool_use_id,
    }

    # 将审计日志追加写入 JSONL 文件
    with open("agent-audit.jsonl", "a") as f:
        f.write(json.dumps(audit_entry) + "\n")

    return {}

# 配置Agent选项，注册后置Hook
options = ClaudeAgentOptions(
    hooks={
        "PreToolUse": [...], # 此处可接前面的安全拦截Hook

        "PostToolUse": [
            HookMatcher(matcher="Write", hooks=[auto_format_on_write]),
            HookMatcher(matcher="Edit", hooks=[auto_format_on_write]),
            HookMatcher(matcher="*", hooks=[audit_all_tools]),
        ]
    }
)

```

8.7.3 Agent SDK Hooks与Shell Hooks的关系

表8-3对比了Agent SDK Hooks与Shell Hooks的核心差异，清晰地展示了从配置化脚本到原生编程接口的演进。

表8-3 Shell Hooks与Agent SDK Hooks对比

Hook 机制	定义方式	执行环境	数据交换	异步支持	调试体验	适用场景
Shell Hooks	settings.json配置+Shell脚本文件	独立子进程	通过stdin/stdout传递JSON字符串	不支持(同步阻塞)	困难(依赖echo打印或写入日志文件)	Claude Code交互模式/Headless CLI
Agent SDK	Python/TypeScript原生函数	当前进程	直接通过函数参数和返	原生支持async/await	高效(支持	基于Agent SDK构建的嵌

Hook 机制	定义方式	执行环境	数据交换	异步支持	调试体验	适用场景
Hooks		内	回值对象		IDE断点调试、单步跟踪)	入式应用

如果希望直接使用Claude Code（无论是终端交互模式还是Headless CLI模式），建议采用Shell Hooks；如果正在使用Agent SDK开发自定义应用、集成服务或构建复杂的自动化工作流，建议采用Agent SDK Hooks。

8.8 4道安全防线

Agent SDK构建了由**4道安全防线组成的纵深防御体系**，如图8-2所示。

纵深防御体系

第一道：权限模式

`permission_mode = 'plan'`

全局开关



第二道：工具白名单

`allowed_tools = ['Read', ...]`

工具级别



第三道：canUseTool 回调

在每次工具调用前进行动态逻辑判断

运行时动态



第四道：PreToolUse Hooks

深度检查具体参数、修改输入拦截危险指令

最细粒度

(图8-2 Agent SDK的纵深防御体系)

第一道：权限模式 `permission_mode = 'plan'` 全局开关

第二道：工具白名单 `allowed_tools = ['Read', ...]` 工具级别

第三道：`canUseTool` 回调 在每次工具调用前进行动态逻辑判断 运行时动态

第四道：`PreToolUse Hooks` 深度检查具体参数、修改输入拦截危险指令 最细粒度

`canUseTool` 是 Agent SDK特有的运行时权限回调机制。与功能丰富但复杂的Hooks不同，`canUseTool`强调以二元判断（允许/拒绝）作为实现动态访问控制的最轻量级手段。

以下代码展示了如何构建一个智能的“守门员”，在工具执行前根据具体内容动态决定是否放行。

```
async def can_use_tool(tool_name: str, tool_input: dict) -> dict:
    """运行权限检查回调"""

    # 场景1: 文件写入保护
    # 防止敏感配置被篡改
    if tool_name in ["Write", "Edit"]:
        file_path = tool_input.get("file_path", "")
        if ".env" in file_path or "secrets" in file_path:
            return {
                "allowed": False,
                "reason": "Access to sensitive files denied"
            }

    # 场景2: 网络命令封锁
    # 即使白名单允许了Bash，也要禁止特定的网络操作
    if tool_name == "Bash":
        command = tool_input.get("command", "")
        if any(cmd in command for cmd in ["curl", "wget", "ssh"]):
            return {
                "allowed": False,
                "reason": "Network commands not allowed"
            }

    return {"allowed": True}

options = ClaudeAgentOptions(
    permission_mode="acceptEdits",
    allowed_tools=["Read", "Write", "Edit", "Grep", "Glob"],
    can_use_tool=can_use_tool,
    hooks={
        "PreToolUse": [HookMatcher(matcher="*", hooks=[audit_all_tools])]
```

```
}  
)
```

4道安全防线的协同逻辑：permission_mode确立全局基调，allowed_tools剔除冗余工具，canUseTool在运行时对每次调用实施动态校验，而PreToolUse Hooks则负责最细粒度的参数审查、输入修正及日志记录审计。

8.9 结构化输出：强制JSON Schema

在构建应用时，你通常需要Claude输出结构化数据而非自由文本。Agent SDK通过output_format参数支持基于JSON Schema的验证。请看以下代码示例。

```
from pydantic import BaseModel  
  
class SecurityReport(BaseModel):  
    summary: str  
    issues: list[dict]      # 结构示例: [{"severity", "file", "line",  
"description"}]  
    risk_score: float      # 范围: 0.0~10.0  
  
options = ClaudeAgentOptions(  
    output_format={  
        "type": "json_schema",  
        "schema": SecurityReport.model_json_schema()  
    },  
    max_turns=10,  
    allowed_tools=["Read", "Grep", "Glob"],  
)  
  
async for message in query(prompt="对src/进行安全审查", options=options):  
    if message.type == "result" and message.structured_output:  
        report = SecurityReport.model_validate(message.structured_output)  
  
        # report 是类型安全的Pydantic对象  
        print(f"风险评分: {report.risk_score}")  
        for issue in report.issues:  
            print(f"  [{issue['severity']}] {issue['file']}: {issue.get('line',  
'?')} - {issue['description']}")
```

在TypeScript环境中，你可以利用Zod库实现同等功能。请看以下代码示例。

```
import { z } from 'zod';  
  
const SecurityReport = z.object({
```

```

summary: z.string(),
issues: z.array(z.object({
    severity: z.enum(['critical', 'high', 'medium', 'low']),
    file: z.string(),
    description: z.string(),
})),
riskScore: z.number().min(0).max(10),
});

const options: ClaudeAgentOptions = {
    outputFormat: {
        type: 'json_schema',
        schema: z.toJSONSchema(SecurityReport),
    },
};

```

Claude会自动将输出格式化为符合上述Schema的JSON。如果首次生成的内容未通过验证，Agent SDK将自动触发重试机制（最大重试次数由内部策略控制）；如果超过限制仍未成功，系统将返回`error_max_structured_output_retries`错误。

8.10 实战：构建代码分析Web服务

以下是整合了前面所有知识的完整运行示例——代码分析Web服务。该脚本展示了如何配置Claude Agent，通过流式形式接收分析结果，在Web服务场景中（以命令行模拟）输出实时反馈。

```

#!/usr/bin/env python3
"""代码分析Web服务——完整运行示例"""

import asyncio
import sys
from datetime import datetime
from claude_agent_sdk import query, ClaudeAgentOptions

async def analyze_codebase(directory: str, focus: str = "general"):
    """
    分析指定目录的代码库。

    Args:
        directory: 要分析的本地目录路径
        focus: 分析重点领域 - "security" / "performance" / "quality" / "general"
    """
    # 定义不同关注点的System Prompt
    focus_prompts = {
        "security": "专注于安全漏洞：SQL 注入、XSS、敏感信息硬编码、权限控制缺失等。",

```

```

        "performance": "专注于性能问题：N+1 查询、内存泄漏、冗余计算、缓存策略缺失
等。",
        "quality": "专注于代码质量：命名规范、DRY原则、圈复杂度、测试覆盖率建议等。",
        "general": "进行全面代码审查：涵盖安全性、性能、代码质量及架构合理性。"
    }

options = ClaudeAgentOptions(
    model="claude-sonnet-4-6",
    max_turns=15,
    max_budget_usd=0.50,
    allowed_tools=["Read", "Grep", "Glob"],
    permission_mode="plan",          # 只读模式
    cwd=directory,
    append_system_prompt=focus_prompts.get(focus,
focus_prompts["general"]),
)

# 初始化收集器
output_text = []
tools_used = []
metadata = {}

# 执行流式查询
async for message in query(
    prompt=f"分析当前项目的代码，{focus_prompts.get(focus, '')}。请直接输出
Markdown格式的分析报告。",
    options=options,
):
    if message.type == "assistant":
        for block in message.content:
            if hasattr(block, 'text'):
                output_text.append(block.text)
                # 【关键点】流式输出：在实际 Web 服务中，此处应为 yield SSE 事件
                print(block.text, end="", flush=True)
            elif hasattr(block, 'name'):
                tools_used.append(block.name)

    elif message.type == "result":
        metadata = {
            "session_id": message.session_id,
            "cost_usd": message.total_cost_usd,
            "turns": message.num_turns,
            "duration_ms": message.duration_ms,
            "success": not message.is_error,
        }

```

```

# 返回结构化结果
return {
    "report": "\n".join(output_text),
    "tools_used": tools_used,
    "metadata": metadata,
}

async def main():
    import sys
    directory = sys.argv if len(sys.argv) > 1 else "."
    focus = sys.argv if len(sys.argv) > 2 else "general"

    print(f"分析 {directory} (重点: {focus}) ... \n")
    result = await analyze_codebase(directory, focus)

    print(f"\n\n--- 会话统计 ---")
    print(f"状态: {'成功' if result['metadata']['success'] else '失败'}")
    print(f"费用: ${result['metadata'].get('cost_usd', 0):.4f}")
    print(f"耗时: {result['metadata'].get('duration_ms', 0) / 1000:.1f}s")
    print(f"交互轮数: {result['metadata'].get('turns', 0)}")
    print(f"使用工具: {'', ' '.join(result['tools_used'])}")

if __name__ == "__main__":
    asyncio.run(main())

```

在实际的Web服务架构中，调用query函数返回的异步生成器可直接对接SSE(Server-Sent Events)，实现低延迟的实时数据推送。

以下是一个基于FastAPI的完整示例。该架构允许前端实时渲染Claude的“思考”过程，将传统的“加载动画”替换为动态生成的文本流，显著提升用户体验。

```

from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.post("/api/analyze")
async def analyze(request: AnalyzeRequest):
    async def event_stream():
        async for message in query(prompt=request.prompt, options=options):
            if message.type == "assistant":
                for block in message.content:
                    if hasattr(block, 'text'):
                        yield f"data: {json.dumps({'type': 'text', 'content': block.text})}\n\n"

```

```
        elif message.type == "result":
            yield f"data: {json.dumps({'type': 'done', 'cost':
message.total_cost_usd})}\n\n"

    return StreamingResponse(event_stream(), media_type="text/event-stream")
```

8.11 Agent SDK与Headless CLI：如何选型

虽然Agent SDK与Headless CLI都能让Claude在无人值守的环境下运行，但它们的适用场景有着明确的分界（详见表8-4）。选择错误的工具往往会导致架构过度复杂或功能受限。

表8-4 Agent SDK与Headless CLI的场景选择对照

场景	推荐方式	原因
CI/CD流水线中的单步任务	Headless CLI	零依赖部署，直接嵌入YAML配置，不需要安装Python/Node运行时
Shell脚本自动化	Headless CLI	完美契合管道组合，简单直接
构建Web/桌面AI应用	Agent SDK	支持流式输出(SSE)、精细的状态管理及前端交互
需要自定义工具	Agent SDK	支持@tool装饰器注册本地函数，实现复杂业务逻辑闭环
多轮对话/持续会话	Agent SDK	内置session管理，自动维护上下文历史，不需要手动拼接消息
需要精细控制消息流	Agent SDK	基于异步生成器，可逐块拦截、修改或路由消息
并发管理多个实例	Agent SDK	编程化并发控制(如asyncio.gather)，轻松处理高并发请求

一条有意思的经验法则：**如果你的逻辑可以用一行Shell命令描述，请选用CLI**；如果需要写if/else判断或循环，请选用Agent SDK。

咖哥发言

Agent SDK降低了构建Agent的门槛，但“容易”绝不等于“简单”。将Claude嵌入实际产品时，开发者必须妥善解决以下关键问题：错误重试（应对网络波动与API限流）、成本追踪（监控每次调用的Token消耗）、用户隔离（确保不同用户的对话互不干扰）、响应超时（制定长任务的中断策略）以及熔断机制（在连续失败时自动停止调用）。掌握这些工程细节，是将Agent SDK演示原型升级为生产级Agent的必修课。

本章小结

Agent SDK将Claude Code从单一的终端工具，转型为可嵌入任意应用的AI组件。这一转变依托于两大核心API——query函数和ClaudeAgentOptions。

query函数是Agent SDK的入口，其功能对标Headless模式的claude -p命令，但以异步生成器的形式返回消息流，支持实时处理Claude的每一步输出。该消息流包含5种类型：

- system/init（初始化信号）、
- assistant（Claude的响应，可能包含文本内容及工具调用请求）、
- user（工具执行后的反馈结果）和
- result（任务终止信号，携带成本、轮数、耗时等元数据）。

ClaudeAgentOptions则提供了与Headless CLI参数对等的编程接口，涵盖模型选择、工具权限、权限模式、Prompt控制、会话延续及结构化输出等功能。相较于命令行，编程接口的核心优势在于动态配置能力：开发者可根据用户角色动态调整权限模式，依据任务类型灵活切换模型，或基于上一轮的执行结果实时优化下一轮的Prompt。

自定义工具通过@tool装饰器与进程内MCP服务器实现，赋予Claude调用任意自定义函数的能力，无论是查询数据库、调用内部API，还是发送通知。与此同时，4道安全防线（权限模式、工具白名单、canUseTool回调、PreToolUse Hooks）确保Claude在应用中只做该做的事。

从CLAUDE.md配置、Skills定义，到Hooks拦截、MCP集成，再到Headless模式与Agent SDK的全面掌控，每深入一层，意味着我们距离构建成熟的“产品”而非单一的“工具”更近一步。在第9章中，我们将探讨如何将这些能力打包分发，并深入解析Plugins插件生态。

思考题

1. 如果需要为公司内部构建一个“代码影响分析”工具（输入需求描述，输出受影响的代码模块列表及修改难度评估），你将如何设计其系统架构？在此场景中，应重点利用Agent SDK的哪些核心特性？
2. 自定义工具的description字段是供Claude理解的“操作指南”。一个优质的description应清晰包含哪些信息？一个差的description可能导致什么问题？
3. 4道安全防线（权限模式、工具白名单、canUseTool回调、PreToolUseHooks）之间是什么关系？在你的应用场景中，哪些防线是必需的，哪些可以省略？